

Name: Amrithaa D J

SRN: PES2UG24AM017

GitHub link: https://github.com/PES2UG24AM017/15_connect_four

ChatGPT

I inspected the repository before proposing any change. The README explicitly says Task 1 is limited to completing win detection, while Tasks 2–4 cover termination, AI, and feedback separately.

1. Existing project structure

The repository currently contains:

- main.py — entry point
- game.py — turn/game flow
- board.py — board state, disc drops, and **win detection**
- ai.py — basic AI
- requirements.txt

For Task 1, **only board.py needs to change**.

The game already calls:

```
if self.board.winner(self.turn):
```

after a successful drop, so there is no need to change game.py.

2. Current win-detection implementation

The relevant method in board.py is currently:

```
def winner(self, token):  
    directions = [(0, 1), (1, 0)]  
  
    for r in range(ROWS):  
        for c in range(COLS):  
            if self.grid[r][c] != token:  
                continue
```

```

for dr, dc in directions:
    if all(
        0 <= r + i * dr < ROWS and
        0 <= c + i * dc < COLS and
        self.grid[r + i * dr][c + i * dc] == token
        for i in range(4)
    ):
        return True
return False

```

This is the actual implementation currently present in the repository.

3. What is the defect?

The method currently checks only **two directions**:

`directions = [(0, 1), (1, 0)]`

These mean:

Direction dr, dc		Detects
(0, 1)	same row, next column	Horizontal
(1, 0)	next row, same column	Vertical
(1, 1)	next row, next column	✗ Missing
(1, -1)	next row, previous column	✗ Missing

So the existing code can detect:

X X X X

and:

X

X

X

X

but it cannot detect either diagonal:

X . . .

. X . .

. . X .

. . . X

or:

. . . X

. . X .

. X . .

X . . .

The README specifically requires **both diagonal directions** in addition to horizontal and vertical.

Why does this happen?

The winner() method already has a good generic mechanism: it starts from every cell containing the player's token and checks the next four cells using:

$r + i * dr$

$c + i * dc$

So there is **nothing fundamentally wrong with the checking algorithm**.

The defect is simply that the list of directions is incomplete.

4. Smallest modular fix

Change:

directions = [(0, 1), (1, 0)]

to:

directions = [

(0, 1), # horizontal

(1, 0), # vertical

(1, 1), # diagonal: down-right

(1, -1), # diagonal: down-left

```
]
```

That's the core of the Task 1 fix.

Everything else in `winner()` can remain unchanged.

This is preferable to writing four separate win-checking functions because the existing implementation is already modular: the same `all(...)` logic can handle all four directions.

5. Exact code change

File: `board.py`

Replace only this:

```
directions = [(0, 1), (1, 0)]
```

with this:

```
directions = [  
    (0, 1), # horizontal  
    (1, 0), # vertical  
    (1, 1), # diagonal: down-right  
    (1, -1), # diagonal: down-left  
]
```

Therefore, the complete `winner()` method after the Task 1 change should be:

```
def winner(self, token):  
    directions = [  
        (0, 1), # horizontal  
        (1, 0), # vertical  
        (1, 1), # diagonal: down-right  
        (1, -1), # diagonal: down-left  
    ]  
  
    for r in range(ROWS):  
        for c in range(COLS):
```

```

    if self.grid[r][c] != token:
        continue

    for dr, dc in directions:
        if all(
            0 <= r + i * dr < ROWS and
            0 <= c + i * dc < COLS and
            self.grid[r + i * dr][c + i * dc] == token
            for i in range(4)
        ):
            return True

    return False

```

6. Why shorter sequences still won't count

This part is already handled correctly by the existing code.

The check explicitly does:

```
for i in range(4)
```

so it requires these **four positions** to all contain the same token.

For example, three tokens:

X X X .

will fail because the fourth checked position isn't X.

Similarly:

X

X

X

.

will fail.

So **do not change range(4)**.

Also, a sequence of five or six tokens will still correctly count as a win because it contains at least one group of four consecutive tokens. That's normal Connect Four behaviour.

7. Scope of this change

For **Task 1 only**, I recommend changing **nothing else**.

File	Change?	Reason
board.py	✓ Yes	Add the two missing diagonal directions
game.py	✗ No	Existing call to winner() is sufficient
ai.py	✗ No	AI is explicitly Task 3
main.py	✗ No	Not related to win detection
requirements.txt	✗ No	No dependency required

The existing game flow already calls winner() immediately after a move, so once board.py recognizes the two diagonal directions, the existing interface continues to work.

Final Task 1 change

One file, one logical change:

```
- directions = [(0, 1), (1, 0)]  
  
+ directions = [  
+   (0, 1), # horizontal  
+   (1, 0), # vertical  
+   (1, 1), # diagonal: down-right  
+   (1, -1), # diagonal: down-left  
+ ]
```

That is the smallest change that satisfies the README's Task 1 requirement without touching game termination, AI, input handling, or the other tasks.

```
ROWS, COLS = 6, 7  
class Board:  
    def __init__(self):  
        self.grid = [ "." * COLS for _ in range(ROWS) ]  
    def drop(self, col, token):  
        if not 0 <= col < COLS:  
            return None  
        for r in range(ROWS - 1, -1, -1):  
            if self.grid[r][col] == ".":  
                self.grid[r][col] = token  
                return r  
        return None  
    def full(self):  
        return all(self.grid[0][c] != "." for c in range(COLS))  
    def winner(self, token):  
        directions = [ (0, 1), # horizontal  
                        (1, 0), # vertical  
                        (1, 1), # diagonal: down-right  
                        (1, -1), #
```

diagonal: down-left] for r in range(ROWS): for c in range(COLS): if self.grid[r][c] != token:
continue for dr, dc in directions: if all(0 <= r + i * dr < ROWS and 0 <= c + i * dc < COLS and
self.grid[r + i * dr][c + i * dc] == token for i in range(4)): return True return False give the
complete code with indentation

Sure — the issue is only the indentation inside winner(). Here is the complete corrected
board.py code:

ROWS, COLS = 6, 7

```
class Board:
```

```
    def __init__(self):
```

```
        self.grid = [["."] * COLS for _ in range(ROWS)]
```

```
    def drop(self, col, token):
```

```
        if not 0 <= col < COLS:
```

```
            return None
```

```
        for r in range(ROWS - 1, -1, -1):
```

```
            if self.grid[r][col] == ".":
```

```
                self.grid[r][col] = token
```

```
                return r
```

```
        return None
```

```
    def full(self):
```

```
        return all(self.grid[0][c] != "." for c in range(COLS))
```

```
    def winner(self, token):
```

```
        directions = [
```

```
(0, 1), # horizontal
(1, 0), # vertical
(1, 1), # diagonal: down-right
(1, -1), # diagonal: down-left
]
```

```

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] != token:
            continue

        for dr, dc in directions:
            if all(
                0 <= r + i * dr < ROWS
                and 0 <= c + i * dc < COLS
                and self.grid[r + i * dr][c + i * dc] == token
                for i in range(4)
            ):
                return True

    return False

```

This is the **Task 1-only** change: horizontal and vertical detection remain intact, and both diagonal directions are now detected.

```
PS C:\Users\amrit\15_connect_four> python
```

I have completed and committed Tasks 1 and 2 separately. Now I am working ONLY on Task 3 from the README. Current ai.py is: `import random`
class AI: `def choose_column(self, board, me="O", opponent="X"):` # Starter AI is deliberately basic; students improve it in Task 3. `legal`


```
= [c for c in range(7) if board.grid[0][c] == "."] return random.choice(legal) if legal else None
```

Task 3 requirements: 1. The AI must take an immediate winning move when one exists. 2.

The AI must block an immediate player win when necessary. 3. The AI must never select an illegal/full column. 4. Handle the case where there are no legal columns safely. Please: -

Inspect the existing Board implementation and its `drop()` and `winner()` methods before proposing code. - Do NOT rewrite the entire project. - Do NOT modify `game.py`, `board.py`, or `main.py` unless absolutely necessary. - Keep the existing `AI.choose_column(board, me="O", opponent="X")` interface. - Keep the solution simple and understandable. - Do not add external dependencies. - Do not add persistence. - Do not implement advanced minimax or unrelated AI features; only satisfy Task 3. - Explain the proposed algorithm before giving the final `ai.py`. - Pay particular attention to not permanently modifying the board while testing hypothetical moves. - Ensure that a full column is never returned. After proposing the code, explain how it handles: (a) an immediate AI win, (b) an immediate player threat, (c) full columns, (d) a completely full board.

I inspected the repository you specified, **PES2UG24AM017/15_connect_four**, rather than the earlier repository. The current `ai.py` is still the starter random AI, while `board.py` contains the Task 1 four-direction `winner()` implementation.

1. What the current Board provides

The important parts of your current `board.py` are:

```
def drop(self, col, token):
```

```
    if not 0 <= col < COLS:
```

```
        return None
```

```
    for r in range(ROWS - 1, -1, -1):
```

```
        if self.grid[r][col] == ".":
```

```
            self.grid[r][col] = token
```

```
            return r
```

```
    return None
```

```
drop():
```

- rejects columns outside 0–6
- places a token in the lowest available position
- returns the row where it was placed

- returns None if the column is full.

Your Task 1 winner() checks all four required directions:

(0, 1) # horizontal

(1, 0) # vertical

(1, 1) # down-right diagonal

(1, -1) # down-left diagonal

and requires four consecutive matching tokens.

That makes the existing Board suitable for Task 3 without changing board.py.

2. Current AI defect

Your current ai.py is:

```
import random
```

```
class AI:
```

```
    def choose_column(self, board, me="O", opponent="X"):
```

```
        # Starter AI is deliberately basic; students improve it in Task 3.
```

```
        legal = [c for c in range(7) if board.grid[0][c] == "."]
```

```
        return random.choice(legal) if legal else None
```

It only checks whether the **top cell is empty**, then chooses randomly.

Therefore it does **not** currently:

- look for an immediate winning move
- look for an opponent's immediate winning threat
- prioritize either situation

The existing legal-column check does already prevent selecting a completely full column.

3. Simple algorithm for Task 3

We only need three stages.

Step 1 — Find legal columns

legal = columns whose top cell is empty

If there are none:

return None

This safely handles a completely full board.

Step 2 — Test each legal column for an AI win

For every legal column:

1. Temporarily drop me.
2. Ask the existing board.winner(me).
3. If it returns True, this is the winning move.
4. Undo the temporary move.
5. Return that column.

For example:

Try column 3

↓

temporarily put O there

↓

winner("O")?

↓ YES

undo O

↓

return 3

This reuses your existing drop() and winner() methods instead of duplicating win-detection logic.

Step 3 — Test each legal column for an opponent win

If there is no immediate AI win:

1. Temporarily drop opponent.
2. Check winner(opponent).
3. Undo the temporary move.
4. If it would give the opponent four in a row, return that column.

So the priority becomes:

AI can win?

↓ yes

WIN

↓ no

Opponent can win next?

↓ yes

BLOCK

↓ no

Choose any legal column

This exactly matches the Task 3 requirements without implementing minimax or any additional strategy.

4. Important: temporarily modifying the board

The safest simple approach with your existing interface is:

```
row = board.drop(col, token)
```

Then:

```
board.winner(token)
```

Then immediately undo:

```
board.grid[row][col] = "."
```

drop() tells us the exact row that was changed, so we can restore **that exact cell**.

For example:

Before:

.....

.....

.....

.....

.....

X X X

Try column 3:

.....

.....

.....

.....

. . . O . . .

X X X O . . .

Check winner("O")

Then undo:

.....

.....

.....

.....

.....

X X X

So the actual game board is unchanged after the test.

5. Exact ai.py for Task 3

You only need to replace ai.py.

```
import random
```

```
class AI:
```

```
    def choose_column(self, board, me="O", opponent="X"):
```

```
        # Find columns that are not full.
```

```
        legal = [c for c in range(7) if board.grid[0][c] == "."]
```

```
        # No legal moves means the board is full.
```

```
        if not legal:
```

```
            return None
```

```
        # 1. Take an immediate winning move if one exists.
```

```
        for col in legal:
```

```
            row = board.drop(col, me)
```

```
            if board.winner(me):
```

```
                board.grid[row][col] = "."
```

```
                return col
```

```
            board.grid[row][col] = "."
```

```
        # 2. Block the opponent if they have an immediate winning move.
```

```
        for col in legal:
```

```
            row = board.drop(col, opponent)
```

```
if board.winner(opponent):  
    board.grid[row][col] = "."  
    return col
```

```
board.grid[row][col] = "."
```

```
# 3. Otherwise, choose any legal column.  
return random.choice(legal)
```

Why this is the smallest appropriate change

Only **ai.py** changes.

File	Change
------	--------

ai.py	✓ Task 3 implementation
-------	-------------------------

board.py	✗ No change
----------	-------------

game.py	✗ No change
---------	-------------

main.py	✗ No change
---------	-------------

Your existing interface remains exactly:

```
choose_column(self, board, me="O", opponent="X")
```

and no external dependencies are added.

6. Requirement-by-requirement check

(a) Immediate AI win

Suppose:

```
O O O . . .
```

and column 3 would produce four O's.

The AI temporarily places O there, calls:

```
board.winner(me)
```

which uses your Task 1 four-direction detection, sees the win, undoes the test move, and returns column 3.

Result: AI takes the winning move.

(b) Immediate player threat

Suppose:

X X X

and the player can win by playing column 3.

If the AI cannot win immediately, it tests:

`board.drop(col, opponent)`

and:

`board.winner(opponent)`

It detects that column 3 would give X four in a row.

The temporary X is removed and column 3 is returned.

Result: AI blocks the player.

The AI's own winning move is checked **before** blocking, as required by the priority implied by the Task 3 requirements.

(c) Full columns

The initial legal-column calculation is:

`legal = [c for c in range(7) if board.grid[0][c] == "."]`

A full column has a non-empty top cell, so it isn't included.

Therefore the AI can never return a full column.

Also, the hypothetical tests only run over legal, so they never intentionally test a full column.

(d) Completely full board

If all seven columns are full:

`legal = []`

Then:

